Technische Universität München

Übungen Rechnereinführung WS 2026/27

Adrian Stein – Thomas Lindae – Ari Breitkreuz

---

4 – Version Control (Git)

Aufgaben, welche mit einem (*) markiert sind, sind Bonusaufgaben und sollten nur bearbeitet werden, wenn man die Zeit dafür hat.

4.1 Git Installation

Installieren Sie Git auf Ihrem Rechner:
https://git-scm.com/book/en/v2/Getting-Started-Installing-Git

- Linux: Mit Ihrem Package Manager.
- MacOS: Wenn Sie `git` das erste mal im Terminal versuchen auszuführen, sollten Sie einen Installationsprompt von Xcode bekommen.
- Windows: https://git-scm.com/downloads/win.
  - Bei **„Choosing the SSH executable“** empfehlen wir **„Use external OpenSSH“**.[1]
  - Der Installer installiert auch eine „Git Bash“ welche als eine Bash-ähnliche[2] Shell verwendet werden kann (als alternative zu cmd und Powershell). Für das verwenden von Git (und damit die Aufgaben auf diesem Blatt) ist die Wahl hier effektiv egal.

4.1.1 Git Setup

Git Commits beinhalten unter anderem Informationen zum Author eines Commits. Speziell wird dabei der Name und die E-Mail gespeichert. Zuerst sollten wir also Git diese Informationen mitteilen:

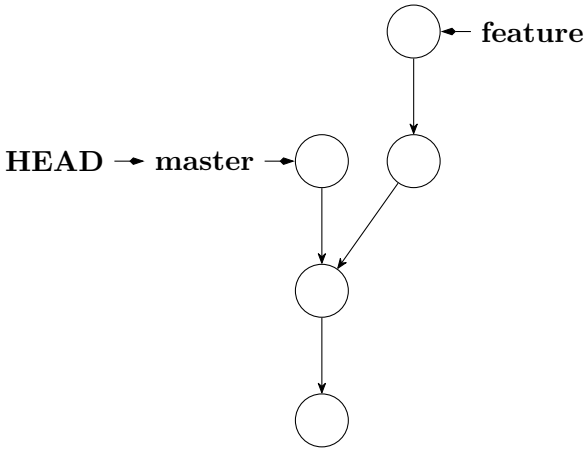
1. `git config --global user.name "Max Mustermann"`
2. `git config --global user.email "max@mustermann.de"`

[1] In „Linux Übung (Teil 2)“ hatten wir bereits SSH Schlüssel mit dem systemweiten OpenSSH, das bei Windows vorinstalliert ist, erstellt. Bei **„Use bundled OpenSSH“** verwendet Git ein eigenes SSH und man müsste wieder neue Schlüssel erstellen.
[2] Als Erinnerung: Bash ist eine der typischen Shells auf Linux.

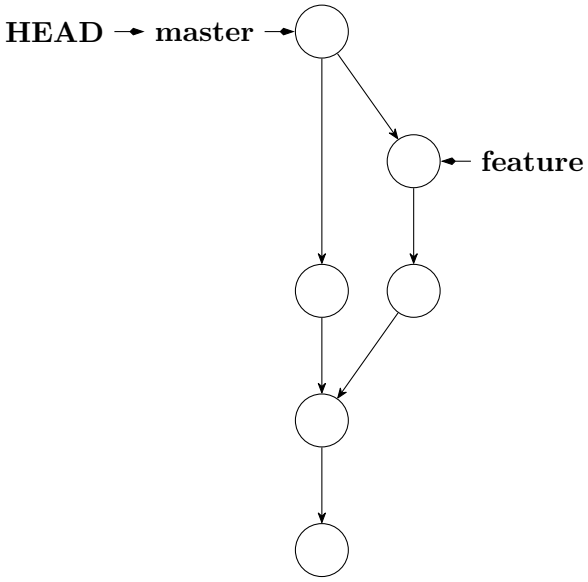
Hier wird kein Account o. Ä. erstellt, der angegebene Text wird einfach nur in Commits mit gespeichert. Die idee ist hier, dass man im Nachhinein die Authoren von Commits identifizieren kann. Weder der Name noch die E-Mail müssen korrekt oder echt sein und man kann beides jederzeit ändern (sollten aber natürlich trotzdem sinnvoll sein).

4.2 Git Lokal

1. Erstellen Sie in einem neuen Ordner ein lokales Git Repository.
2. Erstellen Sie dann Commits und Branches so, dass folgende Git History erzeugt wird:



   Zum Überprüfen könenn Sie folgenden Command verwenden: `git log --graph --all`.
3. Mergen Sie nun den `feature` branch in den `master` branch. Danach sollte Ihre Git History also so aussehen:



4.3 Git GUIs (optional)

Es gibt einige GUIs für Git die Sie verwenden können:

- GitHub Desktop
- GitKraken (Kostenlos mit dem GitHub Student Pack)
- Sourcetree
- Sublime Merge (Kostenpflichtig mit kostenloser Testversion ohne Zeitlimit)
- Und noch viele mehr…

Auch Viele Code Editors haben eine visuelle Git Integration. So gibt es z. B. das VSCode Plugin Git Graph mit dem Sie sich in VSCode Git Graphen visuell anzeigen lassen können.

Wir empfehlen Ihnen die Tools zum visualisieren Ihrer Git Repositories zu verwenden, sich aber trotzdem mit der Git CLI (Command Line Interface) vertraut zu machen.

4.4 Git mit Remotes

Erstellen Sie sich einen Account auf einem Service Ihrer Wahl (GitHub, GitLab, Codeberg oder die GitLab Instanz vom LRZ).

4.4.1 Erstellen eines GitHub/GitLab Repositories

1. Fügen Sie Ihren `ssh` Public Key Ihrem Account hinzu.
2. Erstellen Sie auf dem Service ein neues Repository.
3. Klonen Sie Ihr Repository.
4. Erstellen Sie einen neuen Commit und pushen diesen auf den Remote.

4.4.2 Erster Merge Conflict

Diese Aufgaben sind am besten zu zweit zu lösen. Wenn Sie keinen Partner haben, können Sie stattdessen auch das Repostiory in zwei unterschiedlichen Ordnern auf Ihrem Rechner klonen. Im folgenden wird **S1** und **S2** für die beiden Studenten verwendet.

- Erstellen Sie ein neues Repository auf Ihrem Service.
- Laden Sie ihren Partner zu Ihrem Repository als Developer ein.
- Klonen Sie beide das Repository lokal auf Ihren Rechnern (oder zwei mal an unterschiedlichen Stellen wenn Sie alleine sind).
- **S1:** Machen Sie eine Änderung in dem Repository und Commiten und pushen Sie diese.
- **S2:** Pullen Sie die Änderung Ihres Partners.
- Machen Sie nun beide Änderungen in der gleichen Datei (und Commiten Sie diese). Lassen Sie aber nur **S1** pushen.
- **S2:** Pullen Sie nun die Änderung Ihres Partners. Wenn Sie beide in der gleichen Datei Änderungen gemacht haben, haben Sie nun einen Merge Conflict. Lösen Sie Ihn.

4.5 (*) Gitignore

Wenn Sie an einem Projekt arbeiten, ist es üblich, dass nicht alle Dateien, die Sie lokal in Ihrem Projektordner haben, auch tatsächlich im Git Repository enthalten sein sollten. So sind z. B. Konfigurationsdateien Ihres Editors sehr individuell, die Ihre Mit-Entwickler bei sich nicht haben wollen.

Um nicht bei jedem Commit darauf achten zu müssen, welche Dateien Sie nun tatsächlich in die Staging Area legen, erlaubt Git Ihnen in einer `.gitignore` Datei Pfade und Dateien zu nennen, die Git ignorieren soll.

1. Erstellen Sie einen neuen Ordner, machen Sie den Ordner zu einem Git Repository und fügen Sie eine Datei `.gitignore` mit folgendem Inhalt hinzu:[3]

   ```
   *.out
   ```

2. Erstellen Sie einen initialen Commit mit dieser Datei.

3. Erstellen Sie zwei Dateien, eine `main.txt` und eine `main.out` mit beliebigem Inhalt.

   Betrachten Sie nun den Output von einem `git status` Aufruf.

4.6 (*) Weitere Git Commands

Es gibt noch einige weitere Git commands, wie z. B. `git rebase` oder `git cherry-pick`. Der IBM Blog Post „Learn the workings of Git, not just the commands“ beschreibt manche in weiterem Detail.

Es gibt auch einige Webseiten um Git visuell zu lernen, z. B. `https://learngitbranching.js.org/`.

[3] Der `*` hier ist eine Wildcard. Speziell spezifiziert unsere `.gitignore` alle Dateien, die mit `.out` enden. `https://git-scm.com/docs/gitignore`